

REX Deep Research Agent

Technical Architecture, Workflow Specification & Inter-Agent Protocols
System Version: 2.0.0 · Generated: July 29, 2026

1. Executive Overview

REX (Recursive Exploration eXplorer) is a multi-agent deep research platform designed to automate exhaustive academic, market, and technical intelligence gathering. The system breaks down complex queries, executes parallel web and database retrieval, critically scores source credibility, synthesizes structured arguments with inline citations, and performs iterative evidence gap filling.

Layer	Technologies / Frameworks	Role & Responsibilities
Frontend UI	Next.js 14 (App Router), TypeScript, Tailwind CSS, Framer Motion	Interactive dashboard, progress floor tracking, live telemetry log streaming, PDF/HTML exports.
Backend API	FastAPI, Uvicorn, Python 3.11/3.14, Server-Sent Events (SSE)	Async REST endpoints, session state management, streaming event push.
Agent Orchestration	LangGraph (StateGraph), LangChain Core	Stateful DAG orchestration, message reducers, conditional gap-fill routing.
LLM & Embeddings	Ollama (Qwen 2.5, Phi-3, nomic-embed-text), External API LLM	Sub-question generation, text filtering, section synthesis, quality scoring.
Data & Knowledge	Supabase Vector DB, SQLite (local.db), Knowledge Graph	Past lesson memory retrieval, vector search, lesson extraction, persistent cache.
Scraping & Indexing	DuckDuckGo DDGS, Wikipedia API, Playwright, Jina Reader	Parallel multi-source Web scraping, JavaScript rendering, clean markdown extraction.

2. 9-Stage Multi-Agent Workflow Architecture

The core research pipeline is structured as a 9-node state machine executed deterministically with conditional loopback capabilities:

1. Planner Node (`planner`)

Decomposes the user's research query into 4–8 multidimensional, specific sub-questions based on topic classification (stable technical, emerging trend, company product, policy debate).

2. Memory Retrieval Node (`memory\_retrieval`)

Queries Supabase Vector DB and the local Knowledge Graph using `nomic-embed-text` embeddings to retrieve relevant past research lessons and facts.

3. Searcher Node (`searcher`)

Executes parallel web searches via DuckDuckGo, Wikipedia API, arXiv, and direct HTTP/Playwright scrapers across all sub-question tracks.

4. Filter / Analyzer Node (`filter`)

Filters raw pages using domain blocklists/downweighting rules, chunks text into ~1500 character snippets, and scores relevance using vector embeddings.

5. Synthesis Node (`synthesis`)

Synthesizes extracted evidence chunks into coherent, analytical draft sections addressing each decomposed track.

6. Gap Detector Node (`gap\_detector`)

Audits synthesis completeness. If critical claims lack evidence and iteration count < threshold, triggers a conditional loop back to the Searcher Node.

7. Citation Mapper Node (`citation\_mapper`)

Maps source URLs to bracketed inline numeric citations `[N]` and verifies all reference links against source indices.

8. Report Node (`report\_node\_id`)

Assembles the final comprehensive Markdown report including executive summary, core analysis sections, key statistics, and reference table.

9. Evaluator Node (`evaluator`)

Evaluates final report on 5 quality dimensions (Relevance, Depth, Novelty, Coherence, Citation Accuracy), extracts a 'Lesson Learned', and updates knowledge base.

3. Inter-Agent Communication Protocols

Inter-agent communication in REX is implemented via **LangGraph State-Driven Message-Passing** using a unified `AgentState` schema:

```
class AgentState(TypedDict):
    query: str # Original research query
    sub_questions: List[str] # Decomposed tracks from Planner
    raw_pages: Dict[str, str] # Scraped text from Searcher
    scored_chunks: List[Dict] # Filtered evidence from Filter Node
    synthesis_results: List[Dict] # Track analyses from Synthesis Node
    gap_results: List[Dict] # Target queries from Gap Detector
    gap_iteration: int # Active loop iteration counter
    report: str # Final assembled markdown report
    findings: Annotated[List[str], operator.add] # Accumulative state reducer
    feedback: str # Self-reflection lesson from Evaluator
```

Key Inter-Agent Communication Characteristics:

- **State Mutation & Immutability:** Nodes receive the current `AgentState`, execute processing, and return a dictionary of modified keys. LangGraph merges updates into the graph state.
- **Additive State Reducers:** Accumulator attributes like `findings` and `logs` use `operator.add` reducers to enable non-destructive data aggregation across nodes.
- **Conditional Edge Communication:** The `gap\_detector` node communicates decision signals (`should\_continue\_gap\_fill`) to route execution dynamically to `searcher` or `citation\_mapper`.
- **Client-Server SSE Protocol:** The backend converts internal node transitions into Server-Sent Events (`data: {"node": "filter", "message": "..."}\n\n`) pushed to the Next.js client over HTTP streaming.

4. Data Storage & Knowledge Persistence

- **Supabase Vector DB:** Stores document embeddings and historical lessons evaluated during past runs.
- **Local SQLite Database (`local.db`):** Stores session states, metrics, and offline fallback knowledge data.
- **Redis Cache (`scraper\_cache.db` / **Redis**):** Caches DuckDuckGo and Wikipedia web search results with a 24-hour TTL to prevent redundant network scraping.

5. Quality Evaluation Metrics & Telemetry

Upon report generation, the `Evaluator` node assigns quality scores (0–10) across five dimensions:

Dimension	Evaluation Criteria
Relevance	Direct alignment of synthesized findings with user's core query.
Depth	Comprehensiveness, technical rigor, and presence of quantitative data.
Novelty	Insightfulness of cross-domain analysis and non-trivial findings.
Coherence	Logical flow, structural readability, and clear section hierarchy.
Citation Accuracy	Strict grounding of claims in verified source notes and bracketed links.